Unidad 3

Optimización de código intermedio

Mismo programa, **menos trabajo** al ejecutar

Glosario Unidad 3

Sigla / término	Significado en español
Folding	Plegado de constantes: calcular `2+3` en compilación → `5`
CSE	Eliminar subexpresiones comunes — no calcular dos veces lo mismo
Dead code	Código muerto — instrucciones cuyo resultado nadie usa
BB	Bloque básico — secuencia lineal de triplos sin saltos internos
GFG / CFG	Grafo de flujo de control — diagrama de bloques y saltos
Side effect	Efecto secundario — acción visible (print, escribir memoria)
Compile-time	Tiempo de compilación (antes de ejecutar el .exe)

> **Regla de oro:** optimizar = acortar IR **sin cambiar** lo que el usuario ve al correr el programa.

¿Dónde estamos?

```
AST → IR (U2) → OPTIMIZACIÓN (U3) → ASM (U4)
                       ↑
                misma idea, menos instrucciones
```

Entregable	Archivos
Antes / después	`before.ir` y `after.ir`
Justificación	Tabla: paso | técnica | por qué

¿Qué busca el optimizador?

Objetivo	Ejemplo concreto
Menos instrucciones	`t1 = 2 + 3` → `t1 = 5`
Reutilizar cálculos	segunda suma igual → usar t1 otra vez
Menos temporales	no crear t4 si t3 ya tiene el valor
Menos saltos	simplificar if (avanzado)

No cambia el resultado visible (misma salida, mismos prints).

Local vs global

	Local	Global
Alcance	Un bloque básico	Varios bloques + ramas if/while
Dificultad	Más fácil	Requiere grafo de flujo (GFG)
Técnicas	Folding, CSE local	Propagación de constantes entre bloques
Curso	Empezar aquí	GFG en examen / exposición

Bloque básico (BB)

Secuencia de triplos que se ejecuta **en línea recta** hasta un salto:

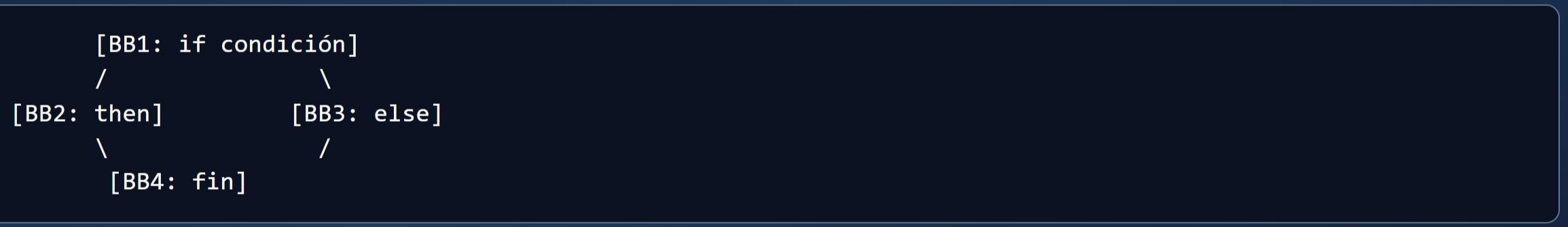
```
t1 = a + b
t2 = t1 * 2        ← mismo bloque
t3 = t2 + 1
ifFalse t3 goto L1  ← última instrucción: sale del bloque
```

Un salto **divide** bloques. El siguiente bloque empieza en L1 o en la línea siguiente.

GFG — Grafo de flujo de control

Nodos = bloques básicos

Flechas = saltos (if, while, goto)



> **GFG** = mapa para ver **por dónde puede pasar** la ejecución.

1. Plegado de constantes (constant folding)

Calcular en **tiempo de compilación** lo que ya es número fijo:

```
ANTES          DESPUÉS
t1 = 2 + 3  →  t1 = 5
t2 = 4 * 5  →  t2 = 20
```

No hace falta que la CPU sume 2+3 cada vez que corre el programa.

2. Propagación de constantes

Si ya sabemos `x = 5`, reemplazar usos de `x`:

```
x = 5
y = x + 1     →    y = 5 + 1    → (folding) → y = 6
```

Global: seguir el valor de `x` a través de varios bloques (cuidado donde dos ramas se unen).

3. CSE — subexpresiones comunes

CSE = *Common Subexpression Elimination*

Si **la misma operación** aparece dos veces:

```
t1 = a + b
t2 = c * d
t3 = a + b    ← repetido
```

Optimizado:

```
t1 = a + b
t2 = c * d
t3 = t1       ← reutilizar, no recalcular
```

4. Eliminación de código muerto

Instrucción cuyo **resultado nadie usa después**:

```
t1 = 2 + 3
t2 = 4 * 5    ← si t2 nunca se usa…
t3 = t1 + 0
x = t3
```

Se puede eliminar la línea de `t2` **si** no hay efecto secundario.

⚠ Cuándo NO eliminar

Situación	¿Eliminar?	Por qué
Temporal no usado	✅ Sí	Sin efecto visible
`call printf(...)`	❌ No	Imprime en pantalla
Escritura a variable global	❌ No	Otro código puede leerla
Entrada del usuario	❌ No	Efecto secundario

> Pregunta de examen típica: **«¿Cuándo NO es código muerto?»** → cuando tiene **efecto secundario**.

Ejemplo integrado

IR inicial:

```
t1 = 2 + 3
t2 = 4 * 5
t3 = t1 + t2
t4 = t1 + t2    ← misma suma que t3
x = t4
```

Paso	Técnica	Resultado
1	Folding	t1=5, t2=20
2	CSE	t4 = t3
3	Asignación	x = t3 (o x = t4 tras CSE)

Tabla de pasos (entregable)

Documenta **cada** transformación:

Paso	Técnica	IR antes	IR después
1	Folding	t1=2+3	t1=5
2	CSE	t4=t1+t2	t4=t3
3	Dead code	(línea extra)	eliminada

El docente califica **correctitud** + **explicación**.

Errores comunes

1. Optimizar y **cambiar el resultado** (bug grave)
2. Eliminar un `call` o I/O pensando que es “muerto”
3. CSE cuando operandos **no son iguales** (ej. `a+b` vs `a+c`)
4. Confundir optimización con **corregir errores semánticos**

Repaso — 5 ideas

1. Optimizar = **mismo significado**, IR más corto
2. **Folding** — calcula constantes al compilar
3. **CSE** — no repetir el mismo cálculo
4. **Dead code** — quitar lo no usado (sin side effects)
5. **BB** y **GFG** — herramientas para optimización global

Próxima: triplos → ensamblador NASM (x86-64)